COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/29/2026	**DATE SUBMITTED:**	8/12/2026
NAME:	Cedric S. Fernandez	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork 4.1 **Array-Based vs. STL Stack Implementations**

ILO.A

CODE

```
#include <iostream>
#include <stack>

int main(){
    std::stack<int> stack1;
    std::cout<<"Testing the stack STL"<<std::endl;
    //isEmpty
    std::cout<<"Is the stack empty? " << stack1.empty()<<std::endl;
    //push
    stack1.push(10);
    std::cout<<"the top of the stack is " << stack1.top() <<std::endl;
    stack1.push(9);
    std::cout<<"the top of the stack is " << stack1.top() <<std::endl;
    stack1.push(8);
    std::cout<<"the top of the stack is " << stack1.top() <<std::endl;
    stack1.push(7);
    std::cout<<"the top of the stack is " << stack1.top() <<std::endl;
    //pop
    stack1.pop();
    std::cout<<"the top of the stack is " << stack1.top() <<std::endl;
    //isEmpty
    std::cout<<"Is the stack empty? " << stack1.empty()<<std::endl;

    //size
    std::cout<< "the size of the stack is: " <<stack1.size()<<
```

```
std::endl;
    //emplace
    stack1.emplace(11);
    std::cout<<"the top of the stack is " << stack1.top() <<std::endl;

    std::stack<int> stack2;
    stack2.push(12);
    stack2.push(13);
    stack2.push(14);
    //swap
    stack1.swap(stack2);
    std::cout<<"the top of the stack1 is " << stack1.top() <<std::endl;
    std::cout<<"the top of the stack2 is " << stack2.top() <<std::endl;

    return 0;
}
```

OUTPUT

```
Testing the stack STL
Is the stack empty? 1
the top of the stack is 10
the top of the stack is 9
the top of the stack is 8
the top of the stack is 7
the top of the stack is 8
Is the stack empty? 0
the size of the stack is: 3
the top of the stack is 11
the top of the stack1 is 14
the top of the stack2 is 11
```

Observation:

This code demonstrates the fundamental operations of a stack container adapter, which follows a LIFO Last-In, First-Out structure. Through things like checking if the stack is empty, viewing the top element and checking total element count, pushing elements onto the stack, constructing them directly on top , and removing the top element, and finally exchanging the entire contents of two separate stacks using

ILO.B

CODE

```
#include <iostream>

//global declarations
#define maxCap 10
int stackArr[maxCap];
int top = -1, newData;
//prototype functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayStack();

int main(){
    //main driver
    int choice;
    while(true){

std::cout<<"================================================================
="<<std::endl;
        std::cout<<"Stack Operations"<<std::endl;
        std::cout<<"1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6.
Display Stack"<<std::endl;

std::cout<<"================================================================
="<<std::endl;
        std::cin>>choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
```

```
            case 3: Top();
            break;
            case 4: std::cout << "is stack empty? "
<<isEmpty()<<std::endl;
            break;
            case 5: std::cout<<"is the stack full?
"<<isFull()<<std::endl;
            break;
            case 6: displayStack();
            break;
            default: std::cout<<"Invalid Choice"<<std::endl;
            break;
        }
    }
}
// function declarations
bool isEmpty(){
    if(top == -1) return true;
    return false;
}
bool isFull(){
    if(top == maxCap - 1) return true;
    return false;
}
void push(){
    if(isFull()){
        std::cout<<"Stack Overflow"<<std::endl;
        return;
    }
    std::cout<<"Enter a new value: "<<std::endl;
    std::cin >> newData;
    stackArr[++top] = newData;
}
void pop(){
    if(isEmpty()){
        std::cout<<"Stack Underflow"<<std::endl;
    }
```

```
    //display the value that we are going to pop
    std::cout<<"Popping: "<<stackArr[top]<<std::endl;
    //decrement the top value from the stack
    top--;
}
void Top(){
    if(isEmpty()){
        std::cout<<"The Stack is Empty"<<std::endl;
        return;
    }
    //check the top value
    std::cout<<"Top element: " << stackArr[top]<<std::endl;
}
void displayStack(){
    if(isEmpty()){
        std::cout<<"The Stack is Empty"<<std::endl;
        return;
    }
    //display stack
    std::cout<<"Stack: ";
    for (int j = top; j >= 0; j--){
        std::cout<< stackArr[j]<<" ";
    }
    std::cout<<std::endl;
}
```

COMPUTER ENGINEERING

QUEZON CITY

OUTPUT

INPUTING 10 VARIABLE

```
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
1
Enter a new value:
1
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
1
Enter a new value:
2
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
1
Enter a new value:
3
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
1
Enter a new value:
4
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
1
Enter a new value:
5
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
1
Enter a new value:
6
```

```
1
Enter a new value:
7
================================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
================================================================
1
Enter a new value:
8
================================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
================================================================
1
Enter a new value:
9
================================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
================================================================
1
Enter a new value:
10
```

POP

```
================================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
================================================================
2
Popping: 10
```

TOP

```
================================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
================================================================
3
Top element: 9
```

isFull

```
================================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
================================================================
5
is the stack full? 0
```

COMPUTER ENGINEERING

QUEZON CITY

Display Stack

```
============================================================
Stack Operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. isFull 6. Display Stack
============================================================
6
Stack: 9 8 7 6 5 4 3 2 1
```

Observation:

This code demonstrates a custom, fixed-size array-based implementation of a stack with a maximum capacity of 10. It manually controls LIFO (Last-In, First-Out) operations using an array index (top). It also include managing cases such as Stack Overflow trying to push into a full array and Stack Underflow trying to pop from an empty array

CONCLUSION

Through this seatwork it demonstrates the fundamental mechanics of the Stack data structure from both a low-level and high-level perspective like building an array-based stack provided hands-on insight into things like fixed memory constraints, and explicit boundary handling like overflow and underflow working with this assignment also highlighted how built-in container adapters simplify code development by managing dynamic sizing and memory automatically.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of content, Grammar correction, Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain function concepts and how to properly use them, along with correcting my grammar for proper explanation and how to do circular Linked list.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.